IPSA
Institut Polytechnique des Sciences Avancées

In422 - Real-Time Information Systems

Final Assignment

Schedule Search

Name : GAUDISSART Gaëtan

Class : 4TS2

Submission date : 28 april 2026

Professor : Jasdeep Singh

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Task Set **2**

3 WCET Measurement of τ_1 **2**
3.1 Methodology 2
3.2 Assumptions 2
3.3 Results 3

4 Schedulability Analysis **4**
4.1 Assumptions 4
4.2 Hyperperiod 4
4.3 Processor Utilization 4
4.4 Workload Test 5
4.5 Response Time Analysis 5
4.6 Computational Complexity 6

5 Non-Preemptive Schedule **6**
5.1 Methodology 6
5.2 Results 7
5.3 Analysis 7
5.4 Gantt Charts 7

6 Schedule Allowing τ_5 to Miss a Deadline **8**
6.1 Results 9
6.2 Analysis 9
6.3 Gantt Charts 9

7 Conclusion **10**

1 Introduction

This report addresses the scheduling analysis of a set of real-time tasks. The objective is to determine the Worst-Case Execution Time (WCET) of task τ_1, verify the schedulability of the task set, and find an optimal non-preemptive schedule that minimizes total waiting time.

2 Task Set

The task set is defined as follows, where C_i is the WCET and T_i is the period (equal to the deadline) of each task :

Task	**C** (ms)	**T** (ms)
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

Table 1: Task set definition

We will do the assuptioms that deadlines are implicit, meaning $D_i = T_i$ for all tasks and all C_i and T_i values are expressed in milliseconds. No unit was specified in the original task set. This assumption is consistent with the WCET measured for τ_1, which is significantly smaller than the other tasks, confirming that milliseconds is the appropriate unit for this task set.

3 WCET Measurement of τ_1

3.1 Methodology

Task τ_1 computes the product of two large randomly generated 64-bit unsigned integers. The execution time was measured using `clock_gettime(CLOCK_MONOTONIC)` in C over 10 000, 50 000 and 100 000 iterations allow us to achieve accuracy on the order of nanoseconds. The raw timing data was then analyzed using Python (NumPy). Three sample sizes were used to observe the evolution of the WCET estimate as the number of samples increases.

3.2 Assumptions

- C_1 is estimated experimentally, not via formal static analysis.
- The absolute maximum was discarded as it reflects OS interruptions (context switches, hardware interrupts) unrelated to τ_1.

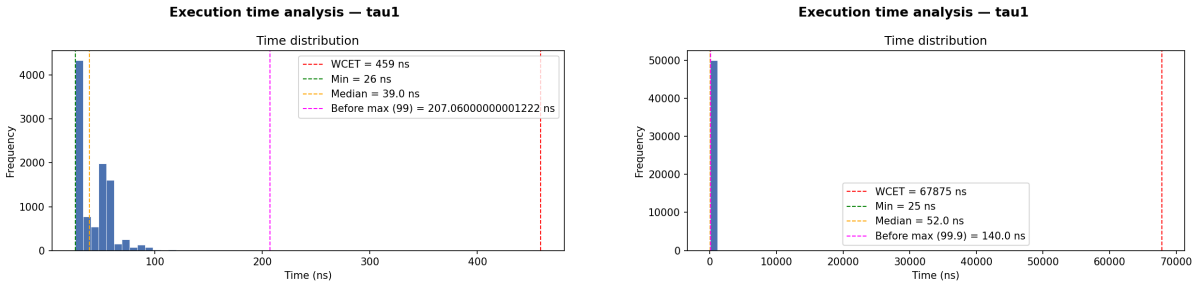
- C_1 is defined as the 99.9th percentile of measured execution times, which provides a conservative yet realistic WCET.

3.3 Results

Statistic	**Value (ns)**
Min	25
Q1	46
Q2	50
Q3	54
Max	42444
Mean	49.982
Standart deviation	151.196
WCET C_1 (99.9th percentile)	**139**

Table 2: Execution time statistics for τ_1 (100 000 samples)

The three histograms below show how the distribution and WCET estimate evolve with the number of samples. As the sample size increases, the 99.9th percentile stabilizes around 139–140 ns, confirming the robustness of the estimate.



(a) 10 000 samples (b) 50 000 samples

Figure 1: Execution time distribution of τ_1 - 10 000 and 50 000 samples

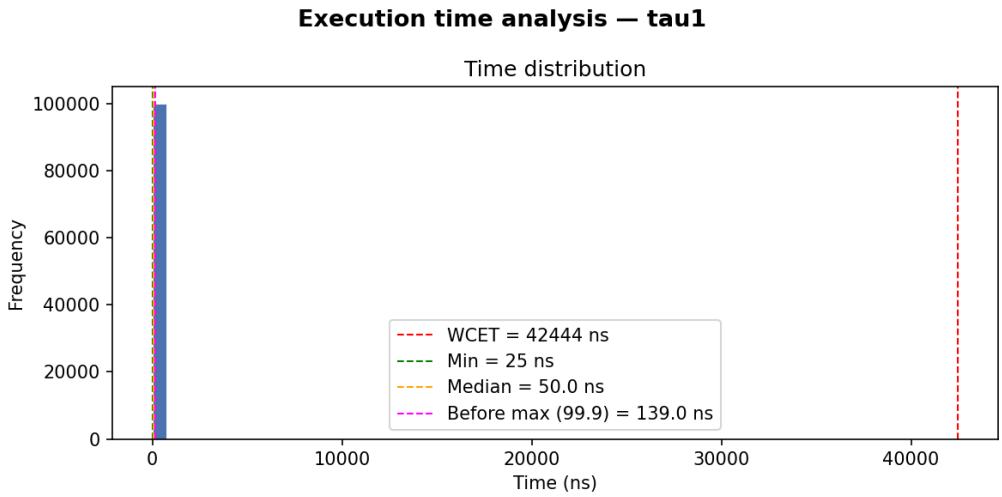


Figure 2: Execution time distribution of τ_1 - 100 000 samples (final estimate)

Thus : $C_1 = 139\,\text{ns} \approx 0.000139\,\text{ms}$

4 Schedulability Analysis

4.1 Assumptions

- We assume a non-preemptive scheduler.
- Tasks are independent
- All tasks are periodic and released at time 0

4.2 Hyperperiod

The hyperperiod H is the Least Common Multiple (LCM) of all task periods. It represents the duration after which the schedule repeats itself.

$$H = \text{LCM}(10, 10, 20, 20, 40, 40, 80) = 80 \text{ ms}$$

This result was confirmed computationally using Python (`numpy.gcd`).

4.3 Processor Utilization

A necessary condition for schedulability is that the total processor utilization U must not exceed 1 :

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

$$U = \frac{0.000139}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = 0.6375139 \leq 1$$

The processor is used at 63.75%, which satisfies the necessary condition. This result was confirmed computationally using Python, yielding $U = 0.6375139000000001$.

4.4 Workload Test

For a non-preemptive schedule, a sufficient condition is that the total work RT over the hyperperiod doesn't exceed H:

$$RT = \sum_{i=1}^{n} \frac{H}{T_i} \times C_i \leq H$$

Task	C_i (ms)	T_i (ms)	H/T_i (jobs)	$\frac{H}{T_i} \times C_i$ (ms)
τ_1	0.000139	10	8	0.001112
τ_2	3	10	8	24
τ_3	2	20	4	8
τ_4	2	20	4	8
τ_5	2	40	2	4
τ_6	2	40	2	4
τ_7	3	80	1	3
			Total RT	**51.001 ms**

Table 3: Response time computation over the hyperperiod

$$RT = 51.001 \text{ ms} \leq H = 80 \text{ ms}$$

This result was confirmed computationally using Python, yielding $RT = 51.001112$ ms. Both conditions are satisfied, thus the task set is schedulable.

4.5 Response Time Analysis

The response time of each job is computed analytically using the following recursive formula, where $a_{ij}^n = (n-1) \times T_i$ is the theoretical arrival time of the n-th job of task τ_i :

$$R_{ij}^n = C_{ij}^n + \max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0)$$

With $R_{ij}^1 = C_i$ for the first job, and since tasks are periodic :

$$a_{ij}^n - a_{ij}^{n-1} = T_i \quad \forall n$$

The formula simplifies to :

$$R_{ij}^n = C_i + \max(R_{ij}^{n-1} - T_i, 0)$$

This analysis is purely theoretical. It assumes each job starts immediately upon arrival, which is an ideal case. In practice, the actual response time depends on the scheduling strategy used, and is verified experimentally in Section 5.

Since $C_i \leq T_i$ for all tasks, we have $R_{ij}^{n-1} - T_i \leq 0$, therefore $\max(R_{ij}^{n-1} - T_i, 0) = 0$ and :

$$R_{ij}^n = C_i \leq T_i = D_i \quad \forall i, n$$

Task	C_i (ms)	T_i (ms)	R_{ij}^n (ms)
τ_1	0.000139	10	0.000139
τ_2	3	10	3
τ_3	2	20	2
τ_4	2	20	2
τ_5	2	40	2
τ_6	2	40	2
τ_7	3	80	3

Table 4: Theoretical response time per task - all jobs respect their deadline

All theoretical response times satisfy $R_{ij}^n \leq D_i = T_i$, confirming that the task set is schedulable. The practical verification is performed in Section 5.

4.6 Computational Complexity

The computational complexity of the scheduling algorithm is analyzed as a function of the number of tasks n. Let $N = \sum_{i=1}^{n} \frac{H}{T_i}$ be the total number of jobs generated over the hyperperiod. In our case $N = 29$ jobs.

The scheduling algorithm runs in $O(N^2)$ time because the first loop is executed N times, one job per iteration. And at each step, all available jobs are scanned to select the next one, which takes at most $O(N)$ operations. Thus :

$$\text{Complexity} = O\left(\left(\sum_{i=1}^{n} \frac{H}{T_i}\right)^2\right) = O(N^2)$$

5 Non-Preemptive Schedule

5.1 Methodology

Three non-preemptive scheduling strategies were tested and compared over the hyperperiod $H = 80$ ms :

- **FCFS** (First Come First Served) : jobs executed in order of arrival
- **SJF** (Shortest Job First) : shortest available job executed first
- **LJF** (Longest Job First) : longest available job executed first

Assumption : When multiple jobs have the same arrival time or WCET, the first one in the list is selected as a reference.

5.2 Results

Strategy	Total waiting time (ms)	Total idle time (ms)	Deadlines missed
FCFS	85.003	28.999	**0**
SJF	70.004	28.999	2
LJF	148.000	28.999	2

Table 5: Comparison of scheduling strategies over $H = 80$ ms

5.3 Analysis

- **LJF** produces the highest waiting time (148 ms) and misses 2 deadlines → worst strategy
- **SJF** produces a lower waiting time (70 ms) but misses 2 deadlines → not valid
- **FCFS** is the only strategy that misses no deadlines → optimal valid strategy

Although SJF minimizes waiting time in theory, it fails here because short jobs are systematically prioritized, causing longer jobs (τ_2, τ_7) to miss their deadlines. Thus FCFS is selected as the optimal strategy.

5.4 Gantt Charts

The scheduling simulation results were first saved into CSV files (one per strategy) containing for each job : the task name, arrival time, start time, end time, deadline, waiting time and whether the deadline was missed. These CSV files were then read by a separate Python script to automatically generate the Gantt charts below using the `matplotlib` library. As this type of visualisation was new to me, AI assistance was used to help write the plotting code.

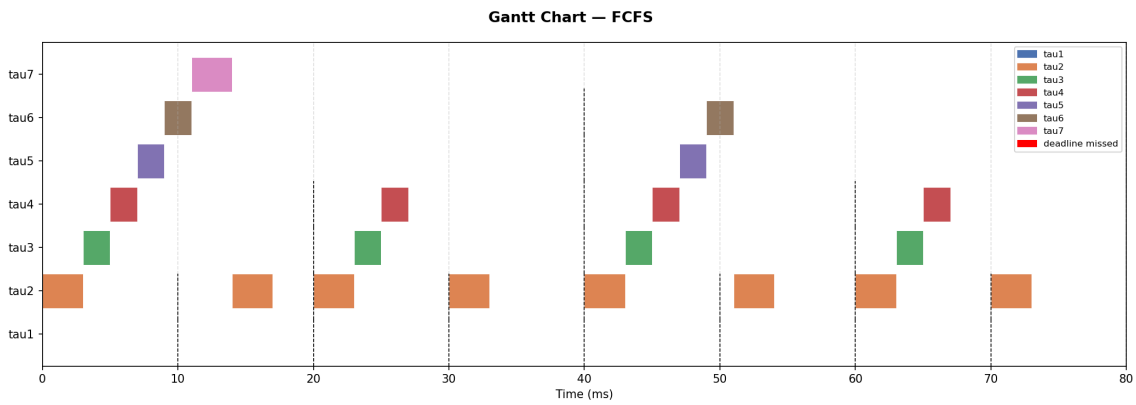


Figure 3: Gantt chart - FCFS (0 deadlines missed)

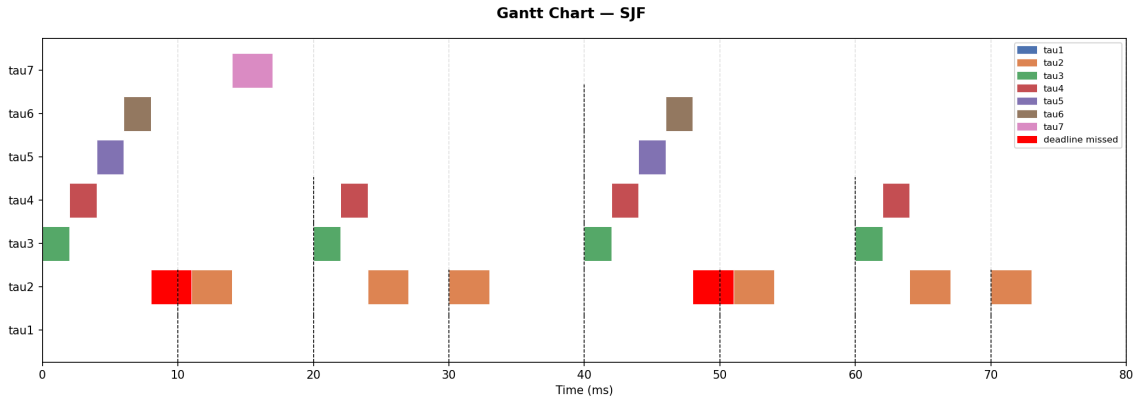


Figure 4: Gantt chart - SJF (2 deadlines missed)

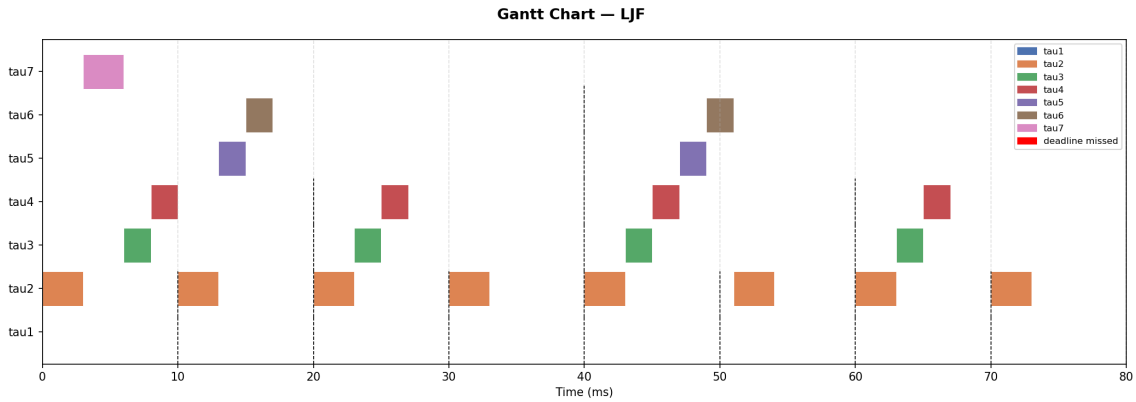


Figure 5: Gantt chart - LJF (2 deadlines missed)

Task τ_1 has a WCET of 0.000139 ms, making its bars invisible on the Gantt charts. However, in the SJF and LJF strategies, τ_1 misses deadlines as confirmed by the simulation results in Table 5.

The theoretical response time analysis in Section 4 showed that $R_{ij}^n = C_i \leq T_i$ for all tasks, suggesting no deadlines would be missed. However, this is a purely theoretical result that assumes each job starts immediately upon arrival. In practice, the non-preemptive scheduler must wait for the current job to finish before starting a new one, which can cause delays. This is why SJF and LJF fail to meet two deadlines even though the theoretical analysis is correct. The scheduling strategy itself introduces a delay that the analytical formula doesn't account for.

6 Schedule Allowing τ_5 to Miss a Deadline

In the previous section, FCFS was selected as the only valid strategy with no missed deadlines. However, the total waiting time of 85.003 ms remains relatively high.

In this section, we relax the constraints by allowing task τ_5 to miss its deadline. This means τ_5 is treated as a low priority task. It is only executed when no other job is available. By doing so, the other tasks can be scheduled more efficiently, potentially reducing the total waiting time.

The same three strategies (FCFS, SJF, LJF) are tested again, but τ_5 is deprioritized. It is only selected when no other job is available in the ready queue. This is still a non-preemptive schedule because τ_5 is not interrupted once started, it simply waits longer before being selected.

6.1 Results

Strategy	**Total waiting time (ms)**	**Total idle time (ms)**	Deadlines missed
FCFS	85.004	28.999	0
SJF	75.004	28.999	0
LJF	139.001	28.999	1

Table 6: Comparison of strategies with τ_5 deprioritized over $H = 80$ ms

6.2 Analysis

- **FCFS** : no change compared to the previous section (85 ms, 0 missed)
- **SJF** : waiting time increased from 70 ms to 75 ms but deadlines missed drops from 2 to 0, which valids this strategy
- **LJF** : waiting time reduced from 148 ms to 139 ms but still misses 1 deadline. The missed deadline belongs to τ_1, not τ_5, because LJF always prioritizes longer jobs, pushing τ_1 (WCET = 0.000139 ms) to execute last, past its deadline of 10 ms. LJF remains incompatible with this task set

Therefore, SJF with τ_5 deprioritized is the optimal strategy. It guarantees no missed deadlines (excluding τ_5) with a total waiting time of 75 ms, which is lower than FCFS (85 ms).

6.3 Gantt Charts

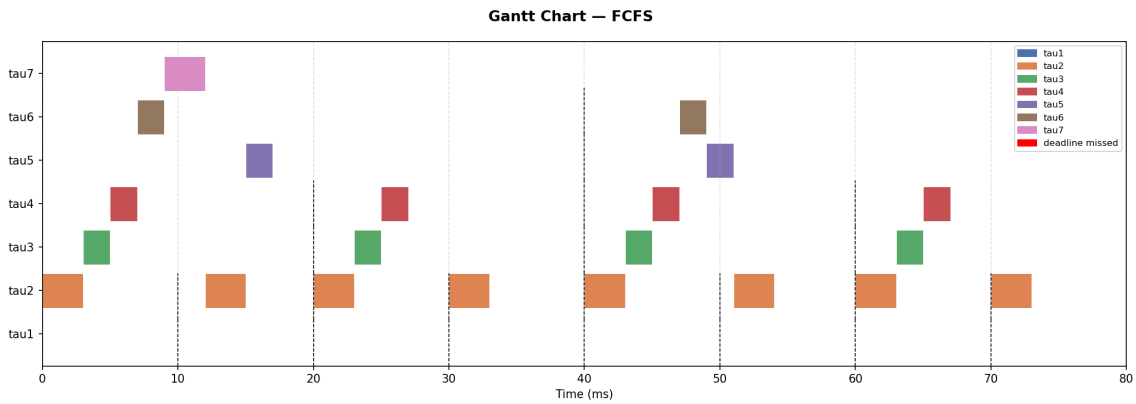


Figure 6: Gantt chart - FCFS with τ_5 deprioritized (0 deadlines missed)

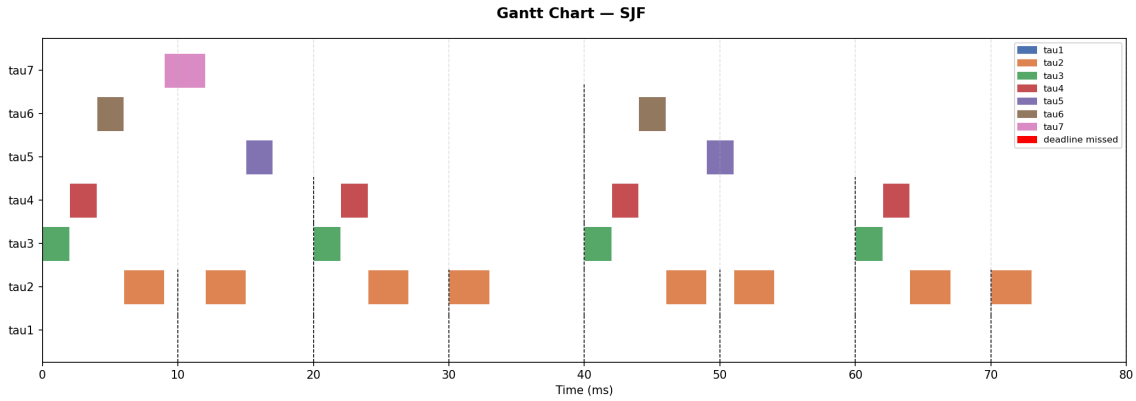


Figure 7: Gantt chart - SJF with τ_5 deprioritized (0 deadlines missed)

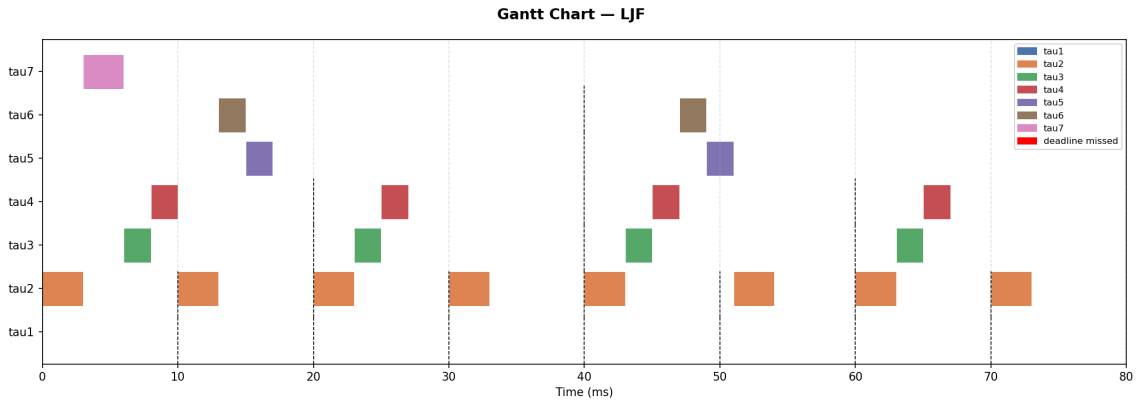


Figure 8: Gantt chart - LJF with τ_5 deprioritized (1 deadline missed - τ_1)

The Gantt charts above were generated automatically using Python (matplotlib library) from CSV files produced by the simulation. As in the previous section, τ_1 bars are invisible due to its WCET of 0.000139 ms.

7 Conclusion

This report presented a complete scheduling analysis of a set of seven real-time tasks. The WCET of τ_1 was first estimated experimentally at $C_1 = 139$ ns using 100 000 measurements, with the 99.9th percentile chosen to discard OS interruption.

The task set was then shown to be schedulable through two conditions. The processor utilization $U = 0.6375 \leq 1$ and total workload $RT = 51.001$ ms $\leq H = 80$ ms. The theoretical response time analysis confirmed that $R_{ij}^n = C_i \leq T_i$ for all tasks.

Three non-preemptive strategies were compared. Without relaxing constraints, only FCFS guaranteed no missed deadlines (waiting time = 85 ms). When τ_5 was allowed to miss its deadline, SJF became the optimal strategy with a lower waiting time of 75 ms and no missed deadlines for the remaining tasks, confirming that relaxing constraints on lower priority tasks can improve overall system performance.

All results were confirmed computationally using Python and C, and visualized through Gantt charts generated automatically from simulation data.